

Othello – Python

Rapport Préliminaire

Grégory Bonzon, Rémi Bousson, Gabriel Ringuet, Matthieu Vigier-Lafosse, Arthur Voisin

30 janvier 2026

1 Explication détaillée du sujet

1.1 Historique et Règles

Le projet consiste à développer le jeu de société Othello, un jeu de stratégie combinatoire abstrait à information parfaite, inventé sous sa forme moderne au XIXe siècle et popularisé dans les années 1970. Le jeu se déroule sur un tablier de 8×8 cases avec 64 pions bicolores, où l'objectif est de posséder la majorité des pions à la fin de la partie. Les Noirs ouvrent toujours le jeu, puis les participants posent tour à tour une pièce de leur couleur en respectant une règle de capture stricte : chaque coup doit obligatoirement « prendre en sandwich » un ou plusieurs pions adverses entre le pion posé et un autre déjà présent, ce qui a pour effet de retourner les pions encadrés pour qu'ils prennent la couleur du joueur actif. Si un joueur ne peut effectuer aucune prise, il doit passer son tour jusqu'à ce que le plateau soit plein ou qu'aucun coup ne soit possible. Une stratégie fondamentale consiste à viser prioritairement les quatre coins du plateau, car un pion placé dans un coin est imprenable et permet de stabiliser durablement ses lignes tout en bloquant les opportunités de l'adversaire.

1.2 Complexité du Jeu

Othello possède une complexité d'espace d'états estimée à environ 10^{28} positions légales, ce qui est inférieur aux échecs (10^{46}) mais reste suffisamment vaste pour nécessiter des algorithmes d'intelligence artificielle performants [1, 2]. Des travaux plus récents sur l'apprentissage automatique, comme le programme Logistello, ont démontré l'importance des stratégies d'ouverture et de fin de partie [3].

2 Algorithmes et Structures de Données

Pour garantir la performance exigée par le cahier des charges (notamment pour l'IA et le mode *Contest*), nous avons choisi des structures optimisées.

2.1 Gestion de l'Histoire : Piles

Pour répondre au besoin d'annulation et de rétablissement des coups (fonctionnalité F12), nous intégrons deux structures de données de type Pile au sein de l'état du jeu.

- **Pile d'Histoire (`undo_stack`)** : Avant chaque modification de l'état du jeu (jouer un coup), une copie de l'état courant est empilée.
- **Pile de Rétablissement (`redo_stack`)** : Lorsqu'un joueur annule un coup, l'état restauré est transféré de la pile d'historique vers cette pile de rétablissement. Cela permet de revenir vers le futur si le joueur change d'avis.

Cohérence : Il est important de noter que si un joueur effectue un *nouveau* coup alors que la pile `redo_stack` n'est pas vide, cette dernière est vidée. En effet, jouer un nouveau coup crée une nouvelle branche dans l'historique, invalidant les états futurs précédemment stockés.

2.2 Représentation du Plateau : Bitboards

Au lieu d'utiliser un tableau à deux dimensions (ex : `int board[8][8]`), nous utiliserons des Bitboards (entiers de 64 bits).

- **Justification** : Cette structure permet de représenter l'ensemble des pions d'un joueur en un seul entier [4] (`uint64`). Les opérations logiques (`AND`, `OR`, `XOR`, `SHIFT`) permettent de calculer les coups légaux et les re-

tournements en parallèle sur tout le plateau, offrant un gain de temps considérable par rapport à une itération case par case.

- **Implémentation** : Le module BitboardOps (voir architecture) gérera les opérations de bas niveau (set_disk, popcount pour le score, décalages pour explorer les 8 directions).

Algorithme de retournement

```

FONCTION compute_flips(own_board, enemy_board,
    x, y)
    // 1. Initialisation
    masque_retournements_total = 0
    // Liste des 8 directions (dx, dy)
    directions = [(-1, -1), (-1, 0), (-1, 1),
                    (0, -1),      (0, 1),
                    (1, -1),  (1, 0),  (1, 1)]
    // 2. Vérification préalable
    SI la case (x, y) n'est pas vide OU hors
        plateau ALORS
        RETOURNER 0
    FIN SI
    // 3. Boucle principale
    POUR CHAQUE (dx, dy) DANS directions FAIRE
        :
        x_courant = x + dx
        y_courant = y + dy
        masque_direction = 0
        TANT QUE est_dans_grille(x_courant,
            y_courant) FAIRE :
            index = (y_courant * 8) +
                x_courant
            bit_mask = 1 << index // Décalage

```

```

    binaire
    // CAS A : Pion ADVERSE
    SI (enemy_board ET bit_mask) != 0
        ALORS
            masque_direction =
                masque_direction OU
                bit_mask
            x_courant = x_courant + dx
            y_courant = y_courant + dy
    // CAS B : Pion À NOUS
    SINON SI (own_board ET bit_mask)
        != 0 ALORS
            masque_retournements_total =
                masque_retournements_total
                OU masque_direction
            ARRÊTER // Passer à la
                direction suivante
    // CAS C : Case VIDE
    SINON
        ARRÊTER
    FIN SI
FIN TANT QUE
FIN POUR
RETOURNER masque_retournements_total
FIN FONCTION

```

2.3 Algorithmes de Jeu

- **Moteur de Règles (Rules)** : Utilisation d'algorithmes basés sur les masques binaires pour get_legal_moves.
- **Minimax & Alpha-Beta** : Pour l'IA standard, nous implémenterons l'algorithme Minimax optimisé par l'élagage Alpha-Beta.
- **Heuristiques** : L'algorithme MinMax ne pouvant explorer l'arbre de jeu jusqu'à la fin (sauf en toute fin de partie), il est nécessaire d'estimer la qualité d'un état intermédiaire. Nous avons détectés trois approches distinctes pour guider l'IA.

2.3.1 1. Évaluation Positionnelle

Cette heuristique statique attribue une valeur à chaque case du plateau (8×8). L'idée est que toutes les positions ne se valent pas au Othello.

- **Logique** : La stratégie vise à occuper les zones stratégiques tout en évitant les pièges.
 - **Les Coins (Poids positif fort)** : Permettent de stabiliser des lignes entières.
 - **Les Cases X et C (Poids négatif)** : Les cases adjacentes aux coins sont dangereuses car elles offrent souvent le coin à l'adversaire.
 - **Les Bords** : Avantageux, mais moins critiques que les coins.
- **Calcul** : Somme des poids des cases occupées par le joueur moins celles de l'adversaire.

2.3.2 2. Évaluation basée sur la Mobilité

Cette fonction mesure la liberté d'action. Au Othello, forcer l'adversaire à ne plus avoir de bons coups (le mettre en manque de coups) est une stratégie dominante.

- **Logique** : Maximiser son nombre de coups légaux tout en minimisant ceux de l'adversaire. Si l'adversaire a peu d'options, il est statistiquement plus probable qu'il soit forcé de jouer un coup désastreux (donnant un coin, par exemple).
- **Calcul** : Différence simple ou normalisée entre le nombre de coups possibles pour le joueur actuel et ceux de l'adversaire.

2.3.3 3. Évaluation Mixte et Dynamique

Une simple heuristique ne suffit pas pour toute la partie. Cette fonction combine plusieurs critères dont l'importance varie selon la phase de jeu (Ouverture, Milieu, Finale). En **début de partie**, on donne la priorité à la **mobilité** et au contrôle du centre. Ensuite en **milieu de partie** l'accent bascule vers le **positionnement** notamment sur les prises de coins. Pour finir, en **fin de partie** on favorise la **stabilité**, c'est à dire le fait qu'un pion ne puisse plus être repris. Des études sur l'apprentissage automatique et les réseaux de neurones ont confirmé l'importance de combiner ces heuristiques de mobilité et de stabilité pour obtenir des stratégies complexes [5, 6].

Algorithme MinMax avec élagage Alpha-Beta

```
FONCTION AlphaBeta(etat, profondeur, alpha,
    beta, joueur_max)
    // 1. Conditions d'arrêt (Feuille de l'
    arbre)
    SI profondeur == 0 OU la_partie_est_finie(
    etat) ALORS
        RETOURNER evaluer_plateau(etat,
            fonction_choisie)
    FIN SI
    // 2. Générer tous les coups possibles
    coups_possibles = generer_coups(etat)
    // Cas particulier : aucun coup possible (
    passe son tour)
    SI coups_possibles est vide ALORS
        RETOURNER AlphaBeta(etat, profondeur -
            1, alpha, beta, NON joueur_max)
    FIN SI
    // 3. Branche MAX (C'est à l'IA de jouer)
    SI joueur_max EST VRAI ALORS
        valeur_max = -INFINI
        POUR CHAQUE coup DANS coups_possibles
            FAIRE :
                nouvel_etat = appliquer_coup(etat,
                    coup)
                valeur = AlphaBeta(nouvel_etat,
                    profondeur - 1, alpha, beta,
                    FAUX)
                valeur_max = MAX(valeur_max,
                    valeur)
        alpha = MAX(alpha, valeur_max)
```

```
    // Coupure Alpha-Beta
    SI beta <= alpha ALORS
        ARRÊTER // on coupe cette
            branche
    FIN SI
    RETOURNER valeur_max
    // 4. Branche MIN (C'est à l'adversaire de
    jouer)
    SINON
        valeur_min = +INFINI
        POUR CHAQUE coup DANS coups_possibles
            FAIRE :
                nouvel_etat = appliquer_coup(etat,
                    coup)
                valeur = AlphaBeta(nouvel_etat,
                    profondeur - 1, alpha, beta,
                    VRAI)
                valeur_min = MIN(valeur_min,
                    valeur)
                beta = MIN(beta, valeur_min)
            // Coupure Alpha-Beta
            SI beta <= alpha ALORS
                ARRÊTER // on coupe cette
                    branche
            FIN SI
        RETOURNER valeur_min
    FIN SI
FIN FONCTION
```

3 Architecture visée pour le projet

L'architecture suit une approche modulaire séparant clairement la logique, l'affichage et le contrôle, inspirée du modèle MVC mais adaptée aux besoins spécifiques.

3.1 Modules Principaux

Common :

- ConfigManager : Gère le chargement/sauvegarde de la configuration (.othellorc) et les arguments en ligne de commande via ArgParser.
- I18nManager : Gère l'internationalisation (fr/en) via gettext.

Game Engine (Model) : C'est le cœur de l'application.

- GameState : Contient l'état courant (2 bitboards : noirs et blancs), le score, et les piles Undo_stack / redo_stack pour la fonctionnalité F12.
- GameRules : Encapsule la logique pure (coups valides, fin de partie).
- HistoryManager : Gère l'historique des coups pour la sauvegarde et le undo/redo.
- GameClock : Gère les horloges des joueurs et le temps de réflexion.
- BitboardOps : Fournit les opérations basiques de manipulation des bitboards (masques, décalages, compages).
- MoveAdvisor : Gère la fonctionnalité de conseil et le calcul des coups suggérés.

User Interface (View) :

- **View (Interface)** : Définit les méthodes `render()`, `start()` et `get_input()`.
- **CLI** : Comprend `CLIShell` (gestion des commandes textuelles) et `CLIRenderer`.
- **GUI** : Basée sur **PyGObject** (GTK 4), gérée par `GUIApp`.
- `ShortcutManager` : Gère les listeners et la configuration des raccourcis clavier.

Orchestrator (Controller) :

- `AppOrchestrator` : Point d'entrée du programme, gère les instanciations possibles et initialise l'ensemble des composants.
- `GameSession` : Configure les parties et récupère les commandes.
- `Interface GameMode` : Gère le mode de jeu et ses contraintes spécifiques.

Player (Model) : La représentation d'un joueur (qu'il soit humain ou IA).

- `Interface Player` : `RealPlayer` et `AIPlayer` qui regroupe les différentes méthodes pour obtenir le meilleur coup.

Artificial Intelligence :

- `SelectionFunctions` : Fonctions de sélection du meilleur nœud utilisées par MCTS.
- `ModelTrainer` : Gestion du machine learning.

Network :

- `Modules` : `GameServer` et `GameClient` pour le jeu en réseau.

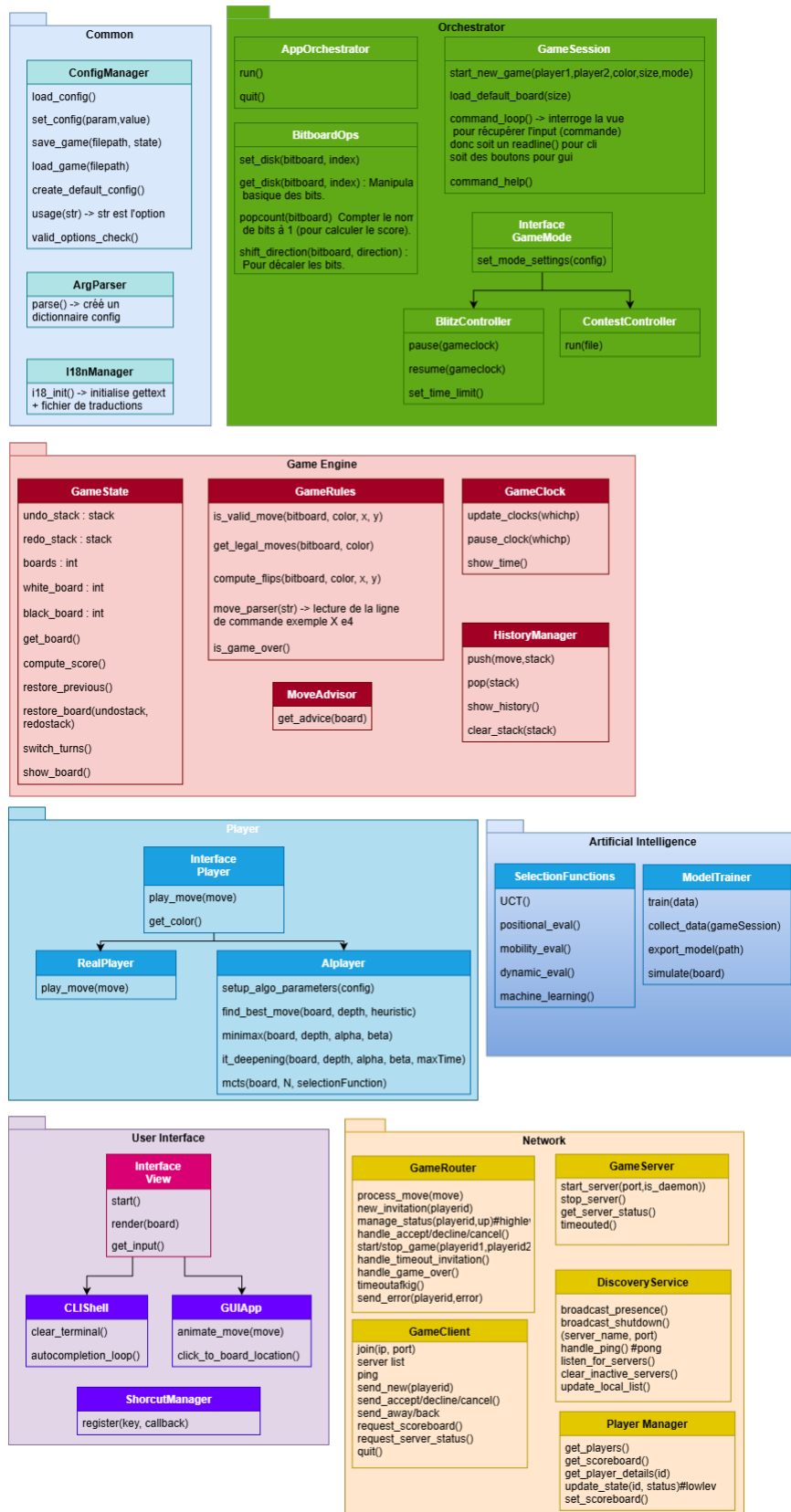


FIGURE 1 – Classes et API

3.2 Réseaux

3.2.1 Initialisation du Serveur

Le démarrage du serveur suit une procédure stricte d'initialisation des ressources et de découverte réseau.

1. **Démarrage (Bootstrap)** : L'administrateur exécute la commande `server start <port>`. Cela déclenche l'ouverture d'un **socket TCP** en mode écoute (listening) pour gérer les futures connexions entrantes.
2. **Instanciation des Gestionnaires** : Le serveur initialise le `PlayerManager` (gestion des états des joueurs connectés) et le `Router` (aiguillage des paquets).
3. **Notification et Découverte** : Une fois prêt, le serveur notifie l'administrateur de son succès. En parallèle, il active un mécanisme de **Broadcast** (diffusion) sur le réseau local. C'est à cet instant précis que le serveur devient visible pour les clients potentiels.

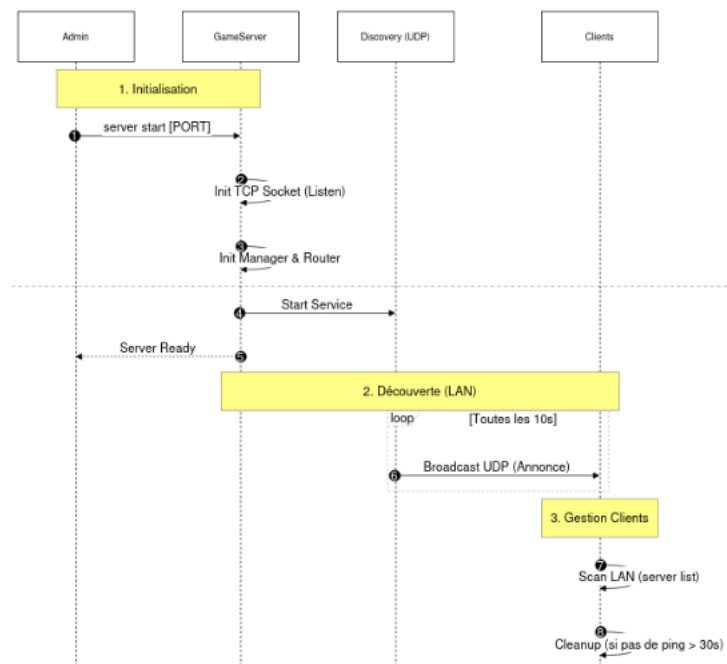


FIGURE 2 – Diagramme de séquence : Initialisation du Serveur

3.2.2 Protocole d'Invitation et Lancement de Partie

Le système permet à un joueur (ex : Alice) de défier un autre joueur connecté (ex : Bob). Ce processus garantit la synchronisation des états avant le début du jeu.

- **Vérification d'état** : Lorsqu'Alice envoie une invitation, le serveur interroge le module `PlayerManager`. Il vérifie que la cible (Bob) est dans l'état `IDLE` (disponible).
- **Négociation** : Si Bob est disponible, la requête lui est transmise. S'il l'accepte, le serveur reçoit la confirmation.
- **Instanciation de la Session** : Le serveur initialise alors les contrôleurs spécifiques à la partie : `GameRouter` et `GameClient`.
- **Début de Match** : Enfin, le serveur envoie un paquet de confirmation aux deux clients, les désignant officiellement comme opposants et lançant la boucle de jeu.

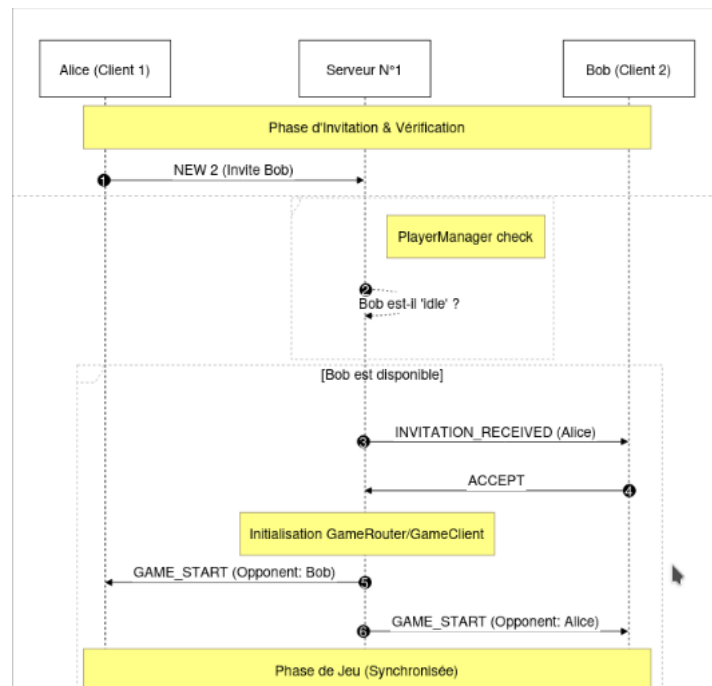


FIGURE 3 – Diagramme de séquence : Processus d'invitation

3.2.3 Cycle de Jeu et Validation (Game Loop)

Une fois la partie lancée, le serveur assure le rôle d'arbitre centralisé pour garantir l'intégrité de la partie. Le diagramme suivant illustre le flux de traitement d'un coup :

1. **Soumission** : Le joueur actif transmet son coup au serveur.
2. **Validation** : Le module `GameRouter` intercepte la requête et sollicite une instance de `GameRule`. Il vérifie la légalité du coup.
3. **Transmission** : Si le coup est valide, le serveur met à jour l'état du jeu et relaie l'information à l'adversaire pour maintenir la synchronisation.
4. **Fin de partie** : Cette boucle se répète jusqu'à la condition de victoire, moment où le serveur notifie le vainqueur aux deux participants.

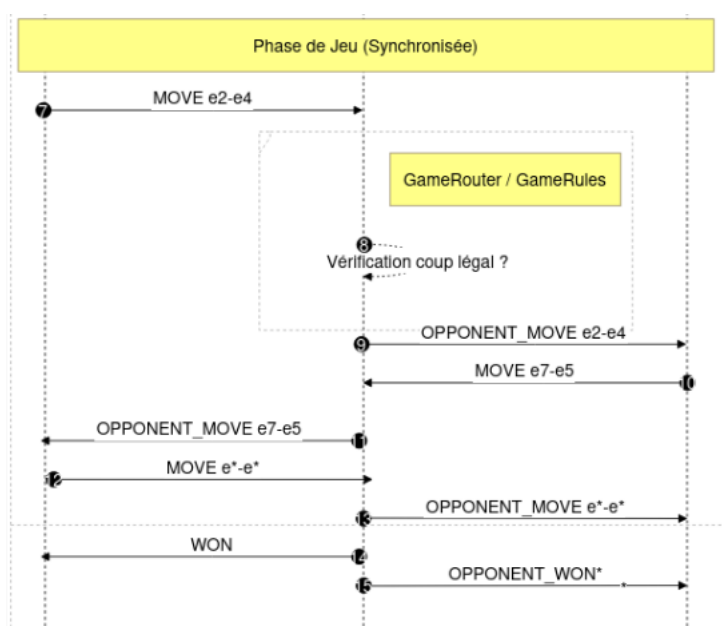


FIGURE 4 – Diagramme de séquence : Déroulement d'un coup

3.2.4 Modèle de Concurrency (Multithreading)

Pour supporter plusieurs parties simultanées sans blocage, le serveur implémente une architecture multithreadée (1 thread pour 2 clients).

- **Isolation des sessions** : Dès que deux joueurs (A et B) s'accordent pour jouer, le serveur instancie un nouveau **Thread de Partie** dédié. Une `GameID` unique est générée et transmise aux clients.
- **Sécurisation des échanges** : Chaque requête client doit inclure le couple (`UserID`, `GameID`). Le thread vérifie systématiquement que le joueur appartient bien à l'instance de jeu ciblée avant de traiter l'action.
- **Cycle de vie** : Le thread reste actif durant toute la durée du match. À la fin de la partie, le thread est détruit pour libérer les ressources et les joueurs sont redirigés vers l'état `IDLE` (lobby principal).

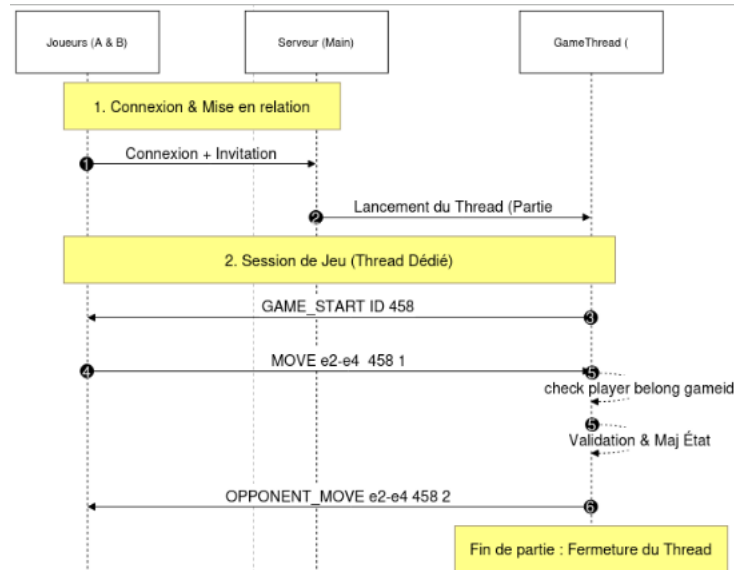


FIGURE 5 – Architecture Multithread (1 Thread / Partie)

4 Spécifications étendues : Conseils aux joueurs (F13)

Besoin Visé : F13. Gestion des conseils (Hint). Le programme doit suggérer un coup pertinent au joueur humain sur demande.

4.1 Sous-besoins et Actions

1. **Calcul du meilleur coup** : Nécessite d'interroger le moteur d'IA sans modifier l'état actuel.
2. **Affichage** : Le conseil doit être affiché clairement (ex : coordonnée en CLI).

4.2 Intégration Architecturale

Création d'une classe `MoveAdvisor` dans le package `GameEngine` avec la méthode `get_advice()`.

Signature : `get_advice`

Contexte : Méthode de la classe `MoveAdvisor` (Package `Model`).

Prototype :

```
def get_advice(board, game_state: GameState) -> tuple[int, int]:
```

Entrées :

- `game_state` : L'état complet du jeu (plateaux et tour actuel).

Sortie :

- `(x, y)` : Tuple d'entiers représentant le meilleur coup suggéré.
- Retourne `(-1, -1)` si aucun mouvement n'est possible.

Description : Cette fonction analyse le plateau actuel avec une profondeur de recherche limitée pour fournir une aide rapide au joueur (commande `hint`), sans modifier l'état réel de la partie.

4.3 Scénario de Validation

Lancer une partie en mode CLI. Puis jouer quelques coups pour arriver à une situation connue (ex : possibilité de prendre un coin). Taper la commande `hint`. Vérifier que le programme propose le coup du coin (coup stratégique fort). Vérifier que l'état du jeu n'a pas changé (pas de coup joué automatiquement).

5 Planning Prévisionnel

Le projet se déroule sur 13 semaines. Voici les grandes phases :

- **Phase 1 : Conception (S0-S3)** Analyse, Architecture.
- **Phase 2 : Développement Core (S4-S5)** Bitboards, Règles, CLI Basique.
- **Phase 3 : Développement Squads (S6-S8)** IA (Rémi et Matthieu), GUI (Gabriel), Réseau (Arthur et Grégory).
- **Phase 4 : Finalisation (S9-S12)** Revue et correction du code.

ID	Phase	Semaine	Tâche (Task)	Responsables
1	CONCEPTION	S0-S1	Analyse Sujet + Architecture (Draw.io) + Git Setup	TOUS
2	CONCEPTION	S2	Algorithmes et Structures de Données + Brouillon Rapport Préliminaire + Pseudo-code IA	TOUS
3	CONCEPTION	S3	Maquettes GUI (Papier/Figma) & Pseudo-code IA & RAPPORT + ORAL PRELIMINAIRE	TOUS
4	DEV CORE	S4	DÉBUT DU CODE: Initialisation Programme & Execution [F1, F2, F3]	Greg & Rémi
5	DEV CORE	S4	DÉBUT DU CODE: Structures & BitboardOps (Macros) [F26, F27]	Gab & Matthieu
6	DEV CORE	S4	DÉBUT DU CODE: GameState & Init Board [F20, F24]	Arthur
T1	TEST UNITAIRES	S4	Mise en place framework de tests (ex: GTest) & Tests des Bitboard Ops Documentation	TOUS
7	DEV CORE	S5	Rules: get_legal_moves & compute_flips (CRITIQUE) [F10, F11, F27]	Gab & Rémi
8	DEV CORE	S5	CLI Basic : Boucle de jeu texte pour tester le moteur & Affichage [F4, F9, F17, F19]	TOUS
T2	TEST UNITAIRES	S5	Tests unitaires du Moteur de Règles (get_legal_moves, compute_flips) Documentation	TOUS
9	MILESTONE	S5	JALON 1: MOTEUR FONCTIONNEL (CLI Basique)	TOUS
10	DEV CORE	S6	CLI Basic : Sauvegarde, Undo/Redo & Shell [F12, F15, F16, F18, F21, F22, F23, F25]	Greg
11	DEV SQUADS	S6	IA: Stratégie Random & Greedy [F8, F13]	Squad IA
T3	TEST UNITAIRES	S6	Tests unitaires des IA (Random & Greedy) Documentation	Squad IA
12	DEV SQUADS	S6	GUI: Fenêtre vide & Affichage Plateau statique [F7, F28]	Squad GUI
13	DEV SQUADS	S6	NET: Sockets Serveur & Client (Base) [F38]	Squad Net
T4	TEST UNITAIRES	S6	Tests unitaires des Sockets & Connexion de base Documentation	Squad Net
14	DEV SQUADS	S7	IA: Minimax (Algo de base) [F31, F32]	Squad IA
T5	TEST UNITAIRES	S7	Tests unitaires de l'IA Minimax Documentation	Squad IA
15	DEV SQUADS	S7	GUI: Gestion Clics Souris & Input [F29, F30]	Squad GUI
16	DEV SQUADS	S7	NET: Échange de coups (Protocole complet) [F39]	Squad Net
17	DEV SQUADS	S8	IA: Alpha-Beta & Heuristiques [F32, F33, F34, F35]	Squad IA
T6	TEST UNITAIRES	S8	Tests unitaires de l'IA Alpha-Beta et des fonctions Heuristiques Documentation	Squad IA
18	DEV SQUADS	S8	GUI: Intégration Réseau dans le Graphique [F28, F40]	Squad GUI + Net
T7	TEST UNITAIRES	S8	Tests unitaires du Protocole Réseau d'échange de coups Documentation	Squad Net
19	DEV SQUADS	S9	IA : MCTS & ML [F36, F37]	Squad IA
20	DEV SQUADS	S9	Mode Tournoi & Tests Charge [F5, F6, F14]	Squad Net
T8	TEST UNITAIRES	S9	Tests unitaires de l'IA MCTS & ML Documentation	Squad IA
21	MILESTONE	S10	JALON 2: FEATURE COMPLETE (BETA)	TOUS
22	POLISH	S10	Animations GUI & Polish Visuel	Squad GUI
23	POLISH	S11	Optimisation Code & Fuites Mémoire	TOUS
T9	POLISH	S11	Revue et correction des tests après optimisation du code	TOUS
24	FINAL	S12	Revue Documentation (Doxygen) & Rapport Final	TOUS
25	FINAL	S14	Soutenance Finale	TOUS

FIGURE 6 – Agenda du Projet

Références

- [1] Paul S. Rosenbloom. A world-championship-level othello program. *Artificial Intelligence*, 19(3) :279–320, 1982.
- [2] Kai-Fu Lee and Sanjoy Mahajan. The development of a world class othello program. *Artificial Intelligence*, 43(1) :21–36, 1990.
- [3] Michael Buro. Logistello : A strong learning othello program. Technical report, NEC Research Institute, 1997.
- [4] Cameron Browne. Bitboard methods for games. *ICGA Journal*, 37(2) :67–84, June 2014.
- [5] David E. Moriarty and Risto Miikkulainen. Discovering complex othello strategies through evolutionary neural networks. *Connection Science*, 7(3-4) :195–209, 1995.
- [6] Kevin Anthony Cherry. An intelligent othello player combining machine learning and game specific heuristics. Master's thesis, Louisiana State University, 2011.